# Universidad de Costa Rica

Facultad de Ingeniería

Escuela de Ciencias de la Computación e Informática

CI 0136

Diseño de Software

Proyecto

*Justificación de decisiones de diseño del MARDA*

| | |
|---|---|
| Josué Amador | B80435 |
| Ignacio Arroyo | B80766 |
| Ricardo Franco | B83050 |
| Gustavo Jiménez | B84060 |
| Daniel Salazar | B87214 |

Grupo 01

Prof. Alan Calderón Castro

Ciudad Universitaria Rodrigo Facio

I Semestre, 2020

## Decisiones de diseño de los principios utilizados

### Principio de responsabilidad única

Las clases creadas tienen un único propósito (FabricaTableros, ElementoTablero, Casilla (genérica), Conector (genérico), Jugador, entre otros).

### Principio de abierto-cerrado

Las clases del MARDA (genéricas) están diseñadas para que se les extienda, y a partir de ahí crear implementaciones específicas para cada juego. Sin embargo, las clases no están pensadas para que se les modifique, porque la idea de este código genérico es que se tengan que adaptar los juegos a ese MARDA y que se facilite la creación. En ese sentido, las clases están cerradas.

### Principio de sustitución de Liskov

Aunque en Python el polimorfismo está demasiado presente, las herencias de las clases de igual forma permitirían que las subclases reemplacen a su clase padre sin ocurrir ningún cambio. Cada clase se comporta de la forma que tenga que hacerlo.

### Principio de inversión de dependencia

En el MARDA, se depende siempre de las clases genéricas (Tablero, ElementoTablero, Casilla), y no de las implementaciones específicas (TableroSE, CasillaSerpiente, entre otros).

## Decisiones de diseño de los patrones arquitectónicos utilizados

### Modelo vista controlador

Este patrón arquitectónico es ideal para juegos que poseen una lógica funcional y que a la vez están programados para controlarse por medio de una interfaz gráfica. En este caso, se implementó con clases VistaClase que representan la versión GUI de una Clase, que corresponde al modelo. La clase Vista funciona como un contenedor de todas las vistas específicas, y la clase Controlador es el controlador propiamente dicho del patrón, que se encarga de manejar la vista y los modelos sin que estos se comuniquen directamente. Es útil para cualquier juego de mesa, asumiendo que todos serán gráficos. Cada equipo de desarrollo puede implementar vistas nuevas a partir de las clases de modelo abstractas (Tablero, Casilla, Jugador…).

# Decisiones de diseño de los patrones de diseño utilizados

## Fábrica abstracta

Se utilizó para la clase Tablero, porque el tablero de un juego de mesa podría tener diversos elementos, los cuales no podemos limitar. Entonces, la clase FabricaTableros permite que se le especialice con una fábrica específica para cualquier juego: FabricaTablerosSE, FabricaTablerosMonopoly, entre otros. Las clases que implementen FabricaTableros deben dar una implementación para los métodos abstractos, y así se asegura que todos los juegos creen tableros apropiados. Estos elementos derivan de una clase abstracta ElementoTablero, que generaliza el tipo de cualquier objeto que se necesite incluir.

## Constructor

Se utiliza en las clases x, y para suplir la necesidad de que otros juegos de mesa utilicen serializaciones distintas a la hora de crear archivos para guardar partidas. Serpientes y escaleras, por ejemplo, concretiza una clase que serializa en formato TXT. Pero esto podría no ser adecuado para juegos como Monopoly, en el que los inventarios suelen ser muy extensos. Por esta razón, este patrón es ideal para que se especialice de acuerdo a las necesidades de un programa, pero siempre con el objetivo de construir un archivo que permita cargar a memoria principal antiguas partidas.

## Estrategia

Se utiliza en las clases x, y para crear funciones que se aplican de manera específica, sin editar el código, aplicando el principio de open-closed. En este caso, se usó para construir en memoria un objeto previamente serializado. La función de deconstrucción de un archivo serializado puede variar dependiendo del formato en el que el juego de mesa lo haya almacenado. Heredando una clase deconstructora, es posible codificar una lectura del archivo sin alterar las clases previamente establecidas, clientes de esta función.